

CAHIER DES CHARGES

SaaS de gestion de scolarité

Recouvrement des frais • Paiements • Communication familles • Reporting

Version 2.0 — Spécification fonctionnelle et technique améliorée

Propriété : EDXSTORE LLC

Date : 16 septembre 2026

Statut : Base de lancement MVP / Vibe Coding

Principe directeur — Construire un produit financier scolaire fiable, multi-tenant, mobile-first et auditable. Priorité au MVP, à la sécurité et aux invariants métier avant les fonctionnalités avancées.

0. Contrôle du document

Élément	Valeur
Version	2.0
Propriétaire	EDXSTORE LLC
Objet	Spécifications fonctionnelles, techniques et critères d'acceptation du SaaS
Cible	Établissements scolaires, équipes financières, parents/tuteurs
Approche	MVP évolutif, architecture modulaire, multi-tenant
Langue principale	Français; anglais prévu dès la conception
Statut	Prêt pour Sprint 0 / architecture et fondations

1. Résumé exécutif

La plateforme est un SaaS B2B2C destiné aux établissements scolaires. Son cœur de valeur est l'automatisation de la facturation scolaire, du suivi des créances, de l'encaissement et de la communication avec les familles. Le produit doit fournir une source fiable de vérité financière, réduire les opérations manuelles et offrir une visibilité temps réel à l'administration et aux parents.

Le MVP ne cherche pas à remplacer immédiatement un ERP scolaire complet. Il se concentre sur les flux à forte valeur : référentiel élèves/familles, frais et factures, échéanciers, paiements et réconciliation, reçus, relances, tableaux de bord, rapports, import/export et sécurité. L'architecture doit toutefois préparer l'extension vers la gestion académique, les applications mobiles, les intégrations ERP et des fonctions IA assistives.

Décision d'architecture — Démarrer en modular monolith full-stack avec frontières de domaines explicites. Les microservices seront envisagés uniquement lorsqu'un besoin mesurable de scalabilité, d'isolation ou de cadence de déploiement le justifie.

2. Contexte, objectifs et limites

2.1 Contexte

- Les établissements suivent encore souvent les frais dans des fichiers Excel, logiciels disparates ou processus semi-manuels.
- Les parents ont besoin d'une vision claire des montants dus, échéances, paiements et reçus.
- La direction et la comptabilité ont besoin d'une traçabilité complète et de rapports consolidés.
- Les canaux de paiement et de notification varient selon les pays; l'application doit donc utiliser des adaptateurs et non dépendre d'un seul fournisseur.

2.2 Objectifs métier

- Réduire le délai moyen de recouvrement et le volume des impayés.
- Augmenter le taux de paiements correctement rapprochés et diminuer les erreurs de saisie.
- Donner aux familles une information fiable, compréhensible et accessible sur mobile.
- Produire des reçus, relevés et rapports vérifiables avec historique d'audit.
- Permettre le pilotage multi-établissements sans mélange de données entre clients.

- Créer une base SaaS monétisable et extensible sans réécriture structurelle.

2.3 Hors périmètre MVP

- Gestion complète des notes, examens, emplois du temps et pédagogie avancée.
- Paie du personnel, comptabilité générale complète et immobilisations.
- Marketplace, e-learning, transport scolaire ou cantine avancée.
- Décisions automatisées de recouvrement par IA sans validation humaine.

3. Acteurs, rôles et responsabilités

Rôle	Responsabilités principales	Niveau d'accès
Super Admin SaaS	Gestion des tenants, plans, support, supervision technique et conformité	Plateforme; accès aux données client limité, tracé et justifié
Admin Établissement	Configuration école, année scolaire, utilisateurs, paramètres, rapports	Tenant complet
Responsable financier / Économe	Frais, factures, échéanciers, réconciliation, rapports et relances	Finance tenant
Caissier	Encaissements manuels autorisés, reçus, consultation des soldes	Finance restreinte
Scolarité / Registrar	Élèves, familles, inscriptions, classes et statuts administratifs	Académique administratif
Parent / Tuteur	Consulter soldes, échéances, payer, télécharger reçus, gérer notifications	Portail famille
Élève	Consultation limitée selon politique de l'établissement	Optionnel MVP / lecture seule
Auditeur / Support autorisé	Consultation contrôlée, temporaire et journalisée	Lecture ciblée

RBAC + permissions — Les rôles sont des profils par défaut. Les autorisations effectives doivent être évaluées par permissions explicites et contexte tenant; aucune vérification ne doit dépendre uniquement de l'interface utilisateur.

4. Périmètre fonctionnel MVP

4.1 Onboarding établissement et configuration

- Création d'un tenant Établissement avec identifiant unique, slug, pays, devise, fuseau horaire et langue.
- Gestion facultative de plusieurs campus/sites au sein d'un même tenant.
- Configuration de l'année scolaire, périodes/trimestres/semestres, niveaux, classes/filières et catégories de frais.
- Invitation des utilisateurs et attribution de rôles/permissions.
- Assistant de démarrage avec checklist de configuration et import initial.

4.2 Élèves, familles et inscriptions

- Création, modification, archivage logique et recherche des élèves.
- Un parent/tuteur peut être lié à plusieurs élèves; un élève peut avoir plusieurs tuteurs avec priorité/contact de facturation.
- Inscription d'un élève à une année scolaire, un niveau/classe/filière; historique conservé.
- Identifiants internes uniques par tenant; prévention des doublons configurable.
- Import CSV/XLSX avec prévisualisation, validation, rapport d'erreurs et reprise partielle.

4.3 Catalogue des frais et facturation

- Création de types de frais : inscription, scolarité, examens, transport, cantine, activités et frais personnalisés.
- Frais uniques ou récurrents; affectation par classe, niveau, filière, élève ou groupe.
- Factures/charges avec lignes détaillées, date d'émission, date d'échéance, statut et solde.
- Remises/escomptes, bourses, exemptions et pénalités avec motif et autorisation.
- Génération en masse contrôlée avec simulation avant publication.

4.4 Échéanciers et créances

- Échéancier standard ou personnalisé par élève/famille.
- Paiements partiels et allocation automatique ou manuelle selon règles documentées.
- États : brouillon, dû, partiellement payé, payé, en retard, annulé; statuts calculés autant que possible à partir du grand livre.
- Aging des créances : courant, 1-30, 31-60, 61-90, >90 jours ou périodes configurables.
- Promesses de paiement et notes internes protégées par permissions.

4.5 Paiements, encaissements et réconciliation

- Paiements en ligne via une couche PaymentGateway; fournisseur réel configurable par marché.
- Adaptateur Mock obligatoire en développement et tests; aucun secret de production dans le code.
- Encaissements hors ligne autorisés : caisse, virement, dépôt, chèque selon établissement.
- Webhooks signés, traitement idempotent, référence externe unique, retries contrôlés et journal des événements.
- Réconciliation : paiement reçu → transaction → allocation → mise à jour du solde → reçu.
- Annulation/correction par reversal ou remboursement; ne jamais supprimer silencieusement une transaction financière validée.
- Reçus numérotés, téléchargeables en PDF, avec possibilité d'envoi par email.

4.6 Relances et communication

- Templates versionnés pour email/SMS/push; français et anglais préparés.
- Scénarios de relance : avant échéance, jour J, retard, escalade et confirmation de paiement.
- Préférences de notification et consentements par tuteur.
- Journal de livraison : queued, sent, delivered, failed, bounced lorsque le fournisseur le permet.
- Mode simulation et quotas pour éviter les envois massifs accidentels.

4.7 Portail parent/tuteur

- Vue consolidée des enfants liés au compte.
- Solde courant, prochaines échéances, historique, détails des frais et paiements.
- Paiement en ligne lorsque disponible; téléchargement de reçus/relevés.
- Mise à jour contrôlée des coordonnées et préférences de notification.
- Interface mobile-first, claire et accessible.

4.8 Reporting et tableaux de bord

- KPIs : total facturé, encaissé, restant dû, taux de recouvrement, impayés, retards, paiements par canal.
- Filtres par période, établissement/campus, classe, niveau, catégorie de frais et statut.
- Rapports : créances, encaissements, journal des transactions, élèves débiteurs, paiements non rapprochés.
- Exports CSV/XLSX et PDF; export volumineux traité en tâche de fond.
- Aucune métrique financière ne doit être calculée à partir de données non rapprochées sans indication explicite.

4.9 Administration, audit et support

- Gestion des utilisateurs, invitations, suspension et rotation des accès sensibles.

- Journal d’audit append-only pour actions critiques : authentification, permissions, frais, factures, paiements, remboursements, exports.
- Support impersonation éventuel seulement avec consentement/politique, durée limitée, bannière visible et audit complet.
- Paramètres tenant : identité visuelle, devise, numérotation, canaux, politique de rétention.

4.10 Abonnement SaaS et monétisation

- Séparer strictement la facturation de l’abonnement SaaS des paiements de scolarité collectés pour les établissements.
- Plans et entitlements conçus dès le schéma : limites utilisateurs/élèves/campus, canaux et fonctions premium.
- Le MVP peut utiliser un plan manuel/free trial; l’intégration de billing SaaS peut être activée ensuite sans refonte du modèle.

5. Règles métier et invariants financiers

- Toutes les tables métier critiques portent tenant_id et sont protégées côté base et côté service.
- Les montants ne sont jamais stockés en nombres flottants binaires. Utiliser NUMERIC/DECIMAL ou unités mineures selon la devise et la règle comptable.
- Une transaction financière validée est immuable dans son sens métier; une correction crée une écriture de reversal/remboursement liée à l’original.
- Chaque webhook de paiement doit être idempotent; une référence fournisseur déjà traitée ne doit pas recréer un paiement.
- Les totaux affichés doivent être dérivables des lignes de facturation, allocations et transactions; éviter les champs de solde modifiables manuellement sans contrôle.
- Les suppressions de données de référence utilisent l’archivage logique lorsqu’un historique financier en dépend.
- Toute remise, pénalité, annulation ou remboursement sensible doit stocker auteur, date, motif et, si configuré, approbateur.
- Le changement d’année scolaire ne doit pas effacer l’historique des inscriptions et soldes précédents.
- Le système doit distinguer created_at, effective_at, paid_at et reconciled_at lorsque ces notions diffèrent.

6. Parcours utilisateurs critiques

ID	Parcours	Résultat attendu
US-01	Admin crée l’établissement, l’année scolaire et invite son équipe	Tenant opérationnel sans accès croisé à d’autres écoles
US-02	Scolarité importe élèves et tuteurs depuis Excel	Données validées, erreurs isolées, rapport d’import disponible
US-03	Économe crée les frais et génère les charges d’une classe	Charges cohérentes, aperçu avant publication et audit
US-04	Parent reçoit un rappel et paie en ligne	Paiement traité une seule fois, allocation correcte et reçu disponible
US-05	Caissier enregistre un paiement hors ligne	Transaction auditée, solde mis à jour, reçu généré
US-06	Responsable finance identifie les impayés >30 jours	Liste filtrée, total fiable, relance ciblée possible
US-07	Admin exporte le journal d’encaissement	Export fidèle aux filtres et traçable
US-08	Auditeur consulte une correction/remboursement	Chaîne complète original → reversal/remboursement visible

7. Exigences non fonctionnelles

Domaine	Exigence MVP / cible
Performance	Pages usuelles < 2,5 s au p75 sur réseau mobile raisonnable; APIs interactives p95 < 800 ms hors dépendances tierces; pagination systématique.
Scalabilité	Architecture stateless côté web; indexation DB; tâches lourdes asynchrones. Les volumes contractuels finaux doivent être validés avant production.
Disponibilité	Objectif initial 99,9 % mensuel pour le service applicatif hors maintenances planifiées; dépend du plan d'hébergement choisi.
Sauvegarde / reprise	Sauvegardes automatisées; restauration testée; objectifs RPO/RTO à contractualiser. Procédure de restauration documentée.
Sécurité	TLS, chiffrement au repos du fournisseur, secrets gérés, MFA recommandé pour admins, rate limiting, validation serveur, protections OWASP.
Accessibilité	WCAG 2.2 AA comme objectif de conception; navigation clavier, focus visible, contrastes, labels et messages d'erreur accessibles.
Internationalisation	FR par défaut; architecture i18n; dates/nombres/devises selon locale; fuseau par tenant.
Compatibilité	Responsive mobile-first; navigateurs modernes; PWA possible ultérieurement.
Observabilité	Logs structurés corrélés, métriques, traces/erreurs, alertes sur webhooks, jobs, échecs de paiement et anomalies.
Confidentialité	Minimisation des données, rétention configurable, export/suppression selon obligations légales et demandes de personnes concernées.

À valider avant Go-Live — Volumes cibles (tenants, élèves, transactions/mois), SLA contractuel, RPO/RTO, pays de commercialisation, règles légales locales, fournisseurs de paiement/SMS et politique de conservation.

8. Architecture technique recommandée

8.1 Stack de référence MVP

Couche	Choix recommandé	Justification
Web full-stack	Next.js 16 App Router + React + TypeScript strict	Rapide à livrer, SSR/Server Components, APIs et UI dans un même produit
UI	Tailwind CSS + shadcn/ui + design tokens	Mobile-first, accessible, cohérent et accélère le Vibe Coding
Validation	Zod + schémas partagés	Contrats explicites entre formulaires, API et domaine
Base de données	PostgreSQL managé (Supabase recommandé)	Transactions relationnelles, contraintes, reporting SQL, RLS
Auth	Supabase Auth ou couche compatible OIDC	Session sécurisée, invitations; possibilité SSO entreprise ultérieure
Stockage	Object storage privé avec URLs signées	Reçus, imports/exports et documents
Jobs	Couche de jobs durable + scheduler abstrait	Relances, exports, réconciliation et retries hors requête web
Email/SMS	Adapters Provider; Resend/Twilio ou fournisseurs locaux configurables	Évite le verrouillage et facilite l'adaptation pays

Couche	Choix recommandé	Justification
Paielements	Interface PaymentGateway + Mock + providers réels par marché	Fiabilité, testabilité et multi-provider
Observabilité	Sentry/OpenTelemetry + logs structurés	Détection et diagnostic des erreurs
Tests	Vitest + Testing Library + Playwright	Unitaires, intégration et E2E
CI/CD	GitHub Actions + environnements preview/staging/prod	Contrôles automatiques avant production
Déploiement	Vercel + DB managée; Docker pour portabilité	MVP rapide avec chemin vers autre cloud si besoin

8.2 Structure logique

- Presentation: App Router, layouts, pages, composants UI et BFF/API.
- Application: use-cases et orchestration; aucune logique financière critique dans les composants React.
- Domain: règles de frais, facturation, paiements, allocation, relances et permissions.
- Infrastructure: DB, auth, email/SMS, paiements, storage, jobs, monitoring.
- Modules recommandés: tenants, identity, academics, people, billing, payments, collections, communications, reporting, audit, saas-billing.

8.3 Multi-tenancy

- Modèle shared database / shared schema avec tenant_id pour le MVP.
- RLS PostgreSQL comme défense en profondeur pour les tables exposées; politiques default-deny lorsque pertinent.
- Le tenant courant est dérivé de l'adhésion utilisateur/session et jamais d'un paramètre client non vérifié.
- Les clés uniques sensibles sont souvent composites avec tenant_id (ex. numéro élève, référence facture tenant).
- Tests automatisés obligatoires d'isolation: un utilisateur du tenant A ne peut lire/modifier aucune donnée du tenant B.

9. Modèle de données conceptuel

Domaine	Entités principales
Tenant	Organization/Tenant, Campus, TenantSettings, SubscriptionPlan, Entitlement
Identité	User, Membership, Role, Permission, Invitation, Session/MFA metadata
Académique	AcademicYear, Term, Program/Track, GradeLevel, ClassGroup, Enrollment
Personnes	Student, Guardian, StudentGuardian, ContactPreference, Address
Facturation	FeeType, FeeSchedule, Charge/Invoice, InvoiceLine, Discount, Penalty, Scholarship
Paielements	PaymentIntent/Request, PaymentTransaction, PaymentAllocation, Refund/Reversal, ReconciliationEvent
Recouvrement	PaymentPlan, Installment, DunningRule, Reminder, CollectionCase, PromiseToPay
Communication	Template, Message, DeliveryAttempt, Consent
Documents	Receipt, Statement, ExportJob, ImportJob, StoredFile
Audit	AuditEvent, SecurityEvent, WebhookEvent, JobExecution

9.1 Contraintes clés

- FK explicites et contraintes NOT NULL sur données structurantes.

- Indexes sur tenant_id + clés de recherche fréquentes (student_id, guardian_id, invoice_id, status, due_date, created_at).
- Unique constraints sur identifiants fonctionnels et références fournisseur.
- Soft delete uniquement lorsque compatible; aucune suppression en cascade destructive des écritures financières.
- Migrations versionnées, revues et testées sur staging avec stratégie rollback/forward-fix.

10. Sécurité, conformité et confidentialité

- Référence de vérification: OWASP ASVS 5.0 comme checklist de sécurité applicative adaptée au niveau de risque.
- Authentification robuste; MFA pour profils administratifs/financiers lorsque disponible; politique de session et révocation.
- Autorisation vérifiée côté serveur pour chaque action sensible; principe du moindre privilège.
- Protection CSRF lorsque applicable, prévention XSS, SQL injection via requêtes paramétrées, validation des uploads et CSP/headers sécurisés.
- Rate limiting sur authentification, invitations, endpoints publics, webhooks et actions d'export/envoi.
- Secrets uniquement via variables d'environnement/secret manager; aucune clé réelle dans Git, logs ou prompts.
- Données sensibles minimisées; logs sans numéros de carte ni secrets; masquage des PII lorsque nécessaire.
- Audit immuable logique des changements financiers et de permissions.
- RGPD: base légale/contrats à définir, droits d'accès/export/suppression lorsque permis, rétention et DPA. Les obligations locales priment selon le pays.
- Procédure d'incident: détection, containment, rotation des secrets, notification selon obligations, post-mortem.

11. API et intégrations

- API interne versionnée pour les domaines critiques; REST privilégié pour simplicité du MVP.
- Contrats d'erreur normalisés avec code métier, message utilisateur et correlation_id.
- Pagination cursor ou page/limit selon cas, filtres validés, limite maximale obligatoire.
- Webhooks sortants signés pour événements futurs (payment.received, invoice.overdue, receipt.created).
- Imports/exports asynchrones pour volumes importants; fichiers privés avec expiration.
- SSO OIDC/SAML réservé aux offres entreprise après MVP sauf contrat prioritaire.
- ERP/comptabilité via exports standards d'abord; connecteurs natifs ensuite selon demande client.

12. UX/UI et design system

- Mobile-first pour le portail parent; desktop optimisé pour finance/admin.
- Navigation par rôle avec dashboard contextualisé; éviter les écrans surchargés.
- Design tokens: couleurs, typographie, espacements, radius, états et composants.
- Tables financières: colonnes configurables, filtres, pagination, export et totaux cohérents.
- Formulaire: validation inline, erreurs compréhensibles, sauvegarde brouillon si pertinent, confirmations pour actions irréversibles.
- États obligatoires: loading, empty, error, permission denied, offline/timeout pour dépendances.
- Aucune information critique représentée uniquement par couleur; accessibilité clavier et lecteur d'écran.

13. Stratégie de tests et qualité

Type	Minimum attendu
Unitaires	Règles métier: calculs de soldes, échéances, allocation, pénalités, permissions, statuts.

Type	Minimum attendu
Intégration DB	Contraintes, transactions, RLS/isolation tenant, migrations, queries critiques.
Contrats/adapters	Paiement Mock + webhooks, email/SMS mocks, storage et jobs.
E2E	Login → onboarding → import → facturation → paiement → reçu → reporting.
Sécurité	Tests d'autorisation négative, rate limit, uploads, secrets, dépendances vulnérables.
Performance	Seed de volume représentatif; pages/queries critiques; absence de N+1.
Régression	Suite exécutée en CI; merge bloqué sur typecheck/lint/tests critiques.
UAT	Scénarios métier signés par un responsable école/finance avant production.

14. DevOps, environnements et exploitation

- Environnements séparés: local, preview, staging, production; données et secrets distincts.
- Branch protection, pull requests et revue obligatoire pour migrations/sécurité/paiements.
- CI: install lockfile, lint, typecheck, unit/integration tests, build, scans pertinents.
- CD: preview par PR; staging automatique; production manuelle ou approuvée.
- Feature flags pour fonctions risquées ou progressives.
- Backups et restauration testée périodiquement; runbooks support et incident.
- Dashboard d'exploitation: erreurs, taux de webhooks, jobs en échec, latence, DB, quotas fournisseurs.

15. Roadmap de réalisation

Phase	Durée indicative	Livrables
Sprint 0 — Fondations	1-2 semaines	Repo, architecture, design system, auth, multi-tenancy, DB migrations, CI, environnements, conventions
Sprint 1 — Référentiels	2 semaines	Année scolaire, classes/filières, élèves, tuteurs, inscriptions, import
Sprint 2 — Facturation	2 semaines	Frais, charges/factures, échéanciers, remises, dashboard de base
Sprint 3 — Paiements	2-3 semaines	Mock gateway, paiements hors ligne, provider réel, webhooks, allocations, reçus
Sprint 4 — Recouvrement	2 semaines	Relances, templates, communication, aging, cas de recouvrement
Sprint 5 — Reporting/Admin	2 semaines	Rapports/exports, audit, permissions fines, paramètres
Hardening & UAT	2-4 semaines	E2E, sécurité, perf, accessibilité, restauration, documentation, formation
Go-Live pilote	1-2 semaines	1-3 établissements pilotes, monitoring, support renforcé, corrections

Estimation — Le planning réel dépend des fournisseurs de paiement, des règles locales, de la profondeur du portail parent et des intégrations. Le pilotage par vertical slices fonctionnelles est préférable à une construction couche par couche.

16. Critères d'acceptation MVP

1. Un admin peut créer/configurer un établissement et ne voir que ses données.
2. Un utilisateur sans permission reçoit un refus côté serveur, même si l'URL/API est appelée directement.
3. Import de données: les lignes valides passent, les erreurs sont expliquées et aucune corruption partielle silencieuse ne se produit.
4. Une classe peut recevoir des frais en masse avec preview, confirmation et audit.
5. Un paiement en ligne rejoué par webhook n'est enregistré qu'une seule fois.
6. Un paiement partiel réduit correctement le solde et apparaît dans le relevé et le dashboard.
7. Une correction financière conserve la trace originale et crée une écriture liée.
8. Les reçus sont générés à partir de transactions validées et identifiables.
9. Les relances respectent les préférences/consentements et évitent les doublons.
10. Les rapports se réconcilient avec les transactions de référence pour une période donnée.
11. Les tests d'isolation tenant, de parcours paiement et de permissions critiques passent en CI.
12. Le produit est utilisable sur smartphone pour le parent et sur desktop pour l'admin/finance.

17. Risques et mitigations

Risque	Impact	Mitigation
Paieement/provider indisponible	Élevé	Adapter interface, retries, statut pending, réconciliation et provider alternatif à terme
Mélange de données entre écoles	Critique	tenant_id, RLS, tests négatifs, revue des requêtes, logs de sécurité
Erreurs de calcul financier	Critique	invariants, DECIMAL/minor units, tests unitaires et intégration, immutabilité/reversal
Import de données médiocres	Élevé	prévisualisation, validation, dry-run, rapports d'erreurs, idempotence d'import
Adoption faible	Moyen	onboarding guidé, UI simple, pilote, formation, support et métriques d'usage
Coût SMS/email	Moyen	quotas, préférences, templates, batch, plan premium et fournisseurs locaux
Scope creep	Élevé	MoSCoW, feature flags, backlog post-MVP, definition of done
Dépendance fournisseur cloud	Moyen	adapters, Postgres standard, export des données, Docker et IaC progressivement

18. Priorisation MoSCoW

Priorité	Contenu
MUST	Multi-tenancy; auth/permissions; élèves/tuteurs/inscriptions; frais/factures; paiements et allocations; reçus; relances; reporting de base; import/export; audit; sauvegardes.
SHOULD	Portail parent complet; paiement provider réel principal; SMS; plans de paiement avancés; SSO pour gros clients; branding tenant.
COULD	PWA/mobile natif; WhatsApp; connecteurs ERP natifs; OCR des reçus; analytics avancés; IA assistive.
WON'T MVP	ERP comptable complet; LMS; notes/examens avancés; paie RH; microservices généralisés; décisions financières autonomes par IA.

19. Extensions post-MVP et fonctions IA

- Assistant de rédaction des relances selon ton, langue et contexte, avec validation humaine.
- Détection d'anomalies de paiement/réconciliation et suggestion de vérifications.
- Prévision de trésorerie et tendances de recouvrement fondées sur données agrégées et explicables.
- Recherche conversationnelle sur rapports autorisés, avec garde-fous tenant et permissions.
- Aucun modèle IA ne reçoit de données personnelles non nécessaires; journaliser les usages sensibles et proposer opt-out/configuration.

20. Livrables

- Code source versionné + README de démarrage et architecture.
- Migrations SQL, seeds de développement et dictionnaire de données.
- UI/design system et écrans du MVP.
- Tests unitaires, intégration, E2E et plan UAT.
- Documentation API et intégrations providers.
- Runbooks: déploiement, rollback, backup/restore, incident et support.
- Guide administrateur établissement et guide parent/tuteur.
- Matrice rôles/permissions et registre des actions auditées.
- Dossier de sécurité/configuration et checklist pré-production.

21. Décisions à confirmer avant production

Question	Décision attendue
Marchés/pays de lancement	Détermine paiements, mobile money, SMS, fiscalité, données et conformité
Fournisseur de paiement principal	Stripe / fournisseur régional / banque / autre; sandbox + webhook requis
Devise(s)	Une devise par tenant au MVP recommandé; multi-devise à préciser
Structure école	Établissement unique vs campus multiples vs groupe scolaire
Canaux de notification	Email obligatoire; SMS/WhatsApp/push selon coût et marché
Volumes contractuels	Tenants, élèves/tenant, transactions/mois, taille imports/exports
Rétention	Durée des données académiques, financières, logs et audit
Abonnement SaaS	Tarification par établissement, élève actif, transaction ou paliers
Portail élève	Inclus MVP ou uniquement parent/tuteur
SSO entreprise	Besoin contractuel au lancement ou post-MVP

22. Règles de construction pour Antigravity / Codex

- Ne pas générer tout le produit en une seule passe. Livrer par vertical slices compilables et testables.
- Avant chaque lot: inspecter le repo, résumer l'état, proposer le plan, puis modifier uniquement les fichiers nécessaires.
- Interdiction de remplacer une fonctionnalité existante fonctionnelle sans justification.
- TypeScript strict; pas de any non justifié; validation serveur obligatoire des entrées.
- Aucun secret ou credential réel dans le repo; fournir .env.example.
- Migrations append-only; ne jamais détruire une DB existante sans instruction explicite.

- Chaque action tenant-scoped doit prouver le tenant et la permission côté serveur.
- Chaque feature terminée doit inclure tests pertinents, états UI, gestion d'erreur et documentation courte.
- À la fin de chaque étape: exécuter lint, typecheck, tests et build; corriger les régressions avant de continuer.
- Conserver un CHANGELOG/implementation log court pour faciliter les sessions Vibe Coding suivantes.

23. Références techniques de cadrage

Référence	Lien	Usage
Next.js Documentation	https://nextjs.org/docs	App Router et développement full-stack React.
OWASP ASVS 5.0	https://owasp.org/projects/asvs	Base de vérification des contrôles de sécurité applicative.
PostgreSQL Row Security Policies	https://www.postgresql.org/docs/current/ddl-rowsecurity.html	Isolation et politiques d'accès ligne par ligne en défense en profondeur.

Annexe A — Prompt de lancement

Le prompt maître de lancement Antigravity/Codex est livré séparément en fichier Markdown afin de pouvoir être copié directement dans l'outil de Vibe Coding. Le démarrage recommandé couvre uniquement Sprint 0 + fondations, puis demande un checkpoint avant les modules métier lourds.